

9주 차 보고서

(2026년 2월 15일 ~ 2026년 2월 28일)

작성자

박신우

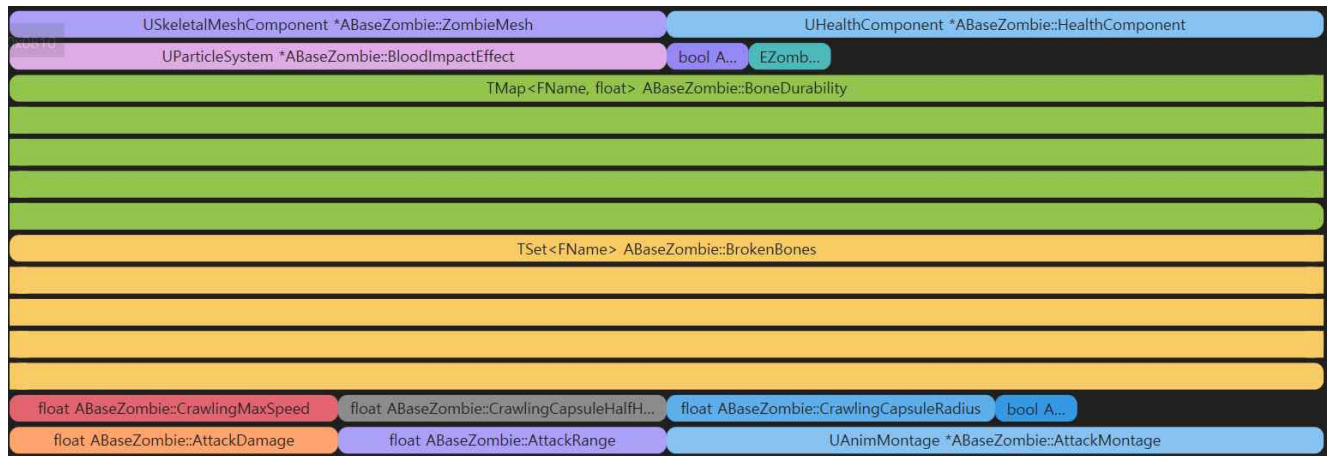
이번 주 한 일

2.18~2.21 클래스 구조 리팩터링

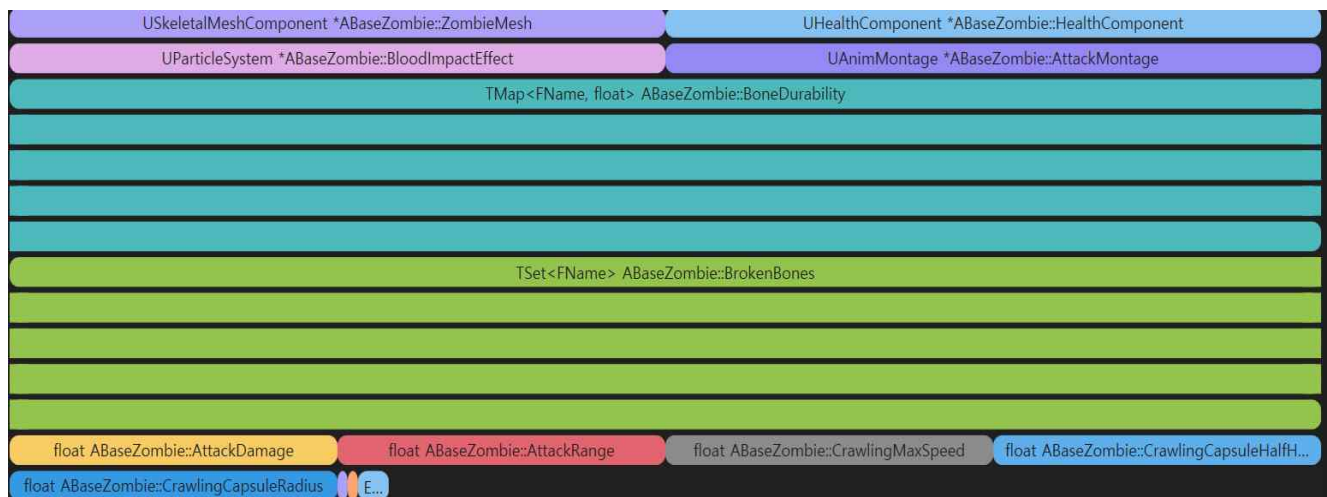
패딩을 입고 공부를 하러 도서관을 가고 있었는데,
문득 1학년때 c++시간에 메모리 패딩을 배웠던 것이 생각이 났다.
패딩 때문에 멤버 변수의 순서가 중요했던 것이 생각이 났는데,
“지금 내가 잘 지키고 있나?”라는 생각이 들었다.
그래서 코드를 확인해 보니 역시나 메모리 얼라인먼트가 개판이었다.

그래서 클래스들의 메모리 얼라인먼트를 고쳐주었다. 아래는 메모리 레이아웃 사진이다.

Before



After



이후에 캡슐화도 진행하여, 멤버 변수를 private 또는 protected로 변경하고,
외부접근은 Getter 와 Setter를 통해서 가능하도록 변경하였습니다.

이번 주 한 일

2.23 행동트리 공격 노드 생성

언리얼의 행동트리에서 호출하면 좀비가 공격하는 기능을 하는 노드를 만들었다.

좀비가 공격이 닿으면 플레이어의 체력이 닳도록 설정을 하였다.

기존 인간이 좀비를 공격할 때와 달리 굳이 HitResult 정보를 보내줄 필요가 없어, 데미지 값만 넘겨주는 ApplyDamage 함수를 HealthComponent에 만들어 주었다.

```
// HitResult 넘겨주지 않는 데미지 함수.  
void UHealthComponent::ApplyDamage(float Damage)  
{  
    if (Damage <= 0.f) return;  
  
    Health -= Damage;  
    Health = FMath::Max(Health, 0.f); // 체력 음수 방지  
  
    UE_LOG(LogTemp, Warning, TEXT("ApplyDamage: %f, Remaining Health: %f"), Damage, Health);  
  
    OnHealthChanged.Broadcast(Health, Damage);  
}
```

정상적으로 데미지가 들어오는 모습을 볼 수 있다.

```
LogTemp: Warning: Zombie BP_BaseZombie_C_2 Attack!  
LogTemp: Warning: Zombie BP_BaseZombie_C_2 Montage!  
LogTemp: Warning: ApplyDamage: 15.000000, Remaining Health: 70.000000  
LogTemp: Warning: Zombie dealt 15.000000 damage!  
LogTemp: Attack Montage Faded
```

2.24 행동트리 추격 노드 생성

좀비가 인간을 추격하는 기능도 노드로 만들어주었다.

기존의 기능을 하던 코드를 `UBTTask_ChaseHuman` 으로 옮겨주었는데,

아래 사진과 같이 한 번만 추격을 하고 도착을 하면 멈추는 문제가 생겼다.



이번 주 한 일

기존 코드에서는 ExcuteTask에서 플레이어에게 좀비가 접근하도록 하였다.
이렇게 하니, 1번만 좀비가 인간을 추적하고 멈추게 되었다.

그래서 TickTask를 만들어서 매 틱마다 좀비가 인간을 추적하도록 만들었다.

생성자에서 bNotifyTick을 참으로 해주고

```
UBTTask_ChaseHuman::UBTTask_ChaseHuman()
{
    NodeName = TEXT("Chase Human");
    // Blackboard 키 초기화
    TargetPlayerKey.SelectedKeyName = FName("TargetPlayer");

    // TickTask 활성화 (매 틱마다 실행)
    bNotifyTick = true;
}
```

ExcuteTask에서 InProgress를 반환해주면

```
// 이동 중이므로 In Progress 반환 (계속 실행)
return EBTNodeResult::InProgress;
```

행동트리가 “아, 이놈 아직 진행중이구나” 하고 TickTask를 호출한다.

TickTask에서 좀비를 추적하고 일정 거리에 도달하면 성공했다고 알려주도록 만들었다.

```
float Distance = FVector::Dist(ZombieCharacter->GetActorLocation(), TargetActor->GetActorLocation());

// 공격 범위에 도달하면 성공 반환 (공격으로 전환)
if (Distance <= StopDistance)
{
    UE_LOG(LogTemp, Warning, TEXT("In attack range! Distance: %f"), Distance);
    FinishLatentTask(OwnerComp, EBTNodeResult::Succeeded);
    return;
}

// 매 틱마다 플레이어 위치 업데이트
AIController->MoveToActor(TargetActor, StopDistance);
```

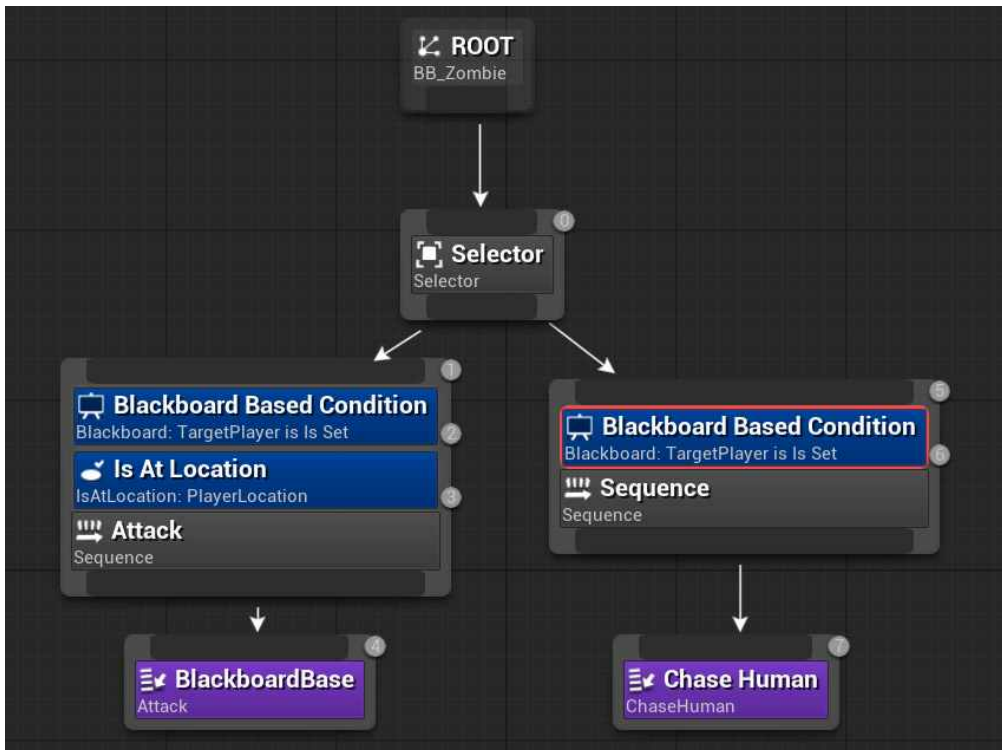
이후 정상적으로 추격하였다.

```
LogTemp: Chasing... Distance: 188.541931
LogTemp: Chasing... Distance: 182.791672
LogTemp: Chasing... Distance: 176.479309
LogTemp: Chasing... Distance: 169.604996
LogTemp: Chasing... Distance: 162.168610
LogTemp: Chasing... Distance: 154.168701
LogTemp: Warning: In attack range! Distance: 145.603729
```

이번 주 한 일

2.25 좀비 행동트리, 좀비 공격 애니메이션

다음과 같이 좀비 행동트리를 구성하였다.



Selector는 왼쪽의 노드부터 평가를 한다.

공격이 가능하면 공격을 하고 공격을 하는 조건이 만족하지않으면, 추격을 하도록 하였다.

플레이어가 감지되고, 공격 범위에 안에 있으면 공격을.

플레이어가 감지되고, 공격 조건이 만족하지않으면 추격을 하도록하였다.

좀비의 공격 애니메이션은 발로 공격하는 애니메이션을 믹사모에서 받아 적용하였다.

차후에 다른 공격 모션들도 넣어서 다양한 공격 모션을 섞어서 사용할 예정이다.

기존 좀비의 이동, 기어다니기 애니메이션은 애니메이션 블루프린트의 상태머신에 묶여있다.

이는 이동속도나 이동상태에 따라서 좀비의 애니메이션이 자연스럽게 변하기 위함이다.

하지만 공격과 같은 모션은 즉각적으로 나와야 해서, 몽타주를 사용해주었다.

걷는 도중에 공격을 해야하면, 즉각적으로 애니메이션이 나와 기존의 애니메이션을 덮어쓰도록 하였다.

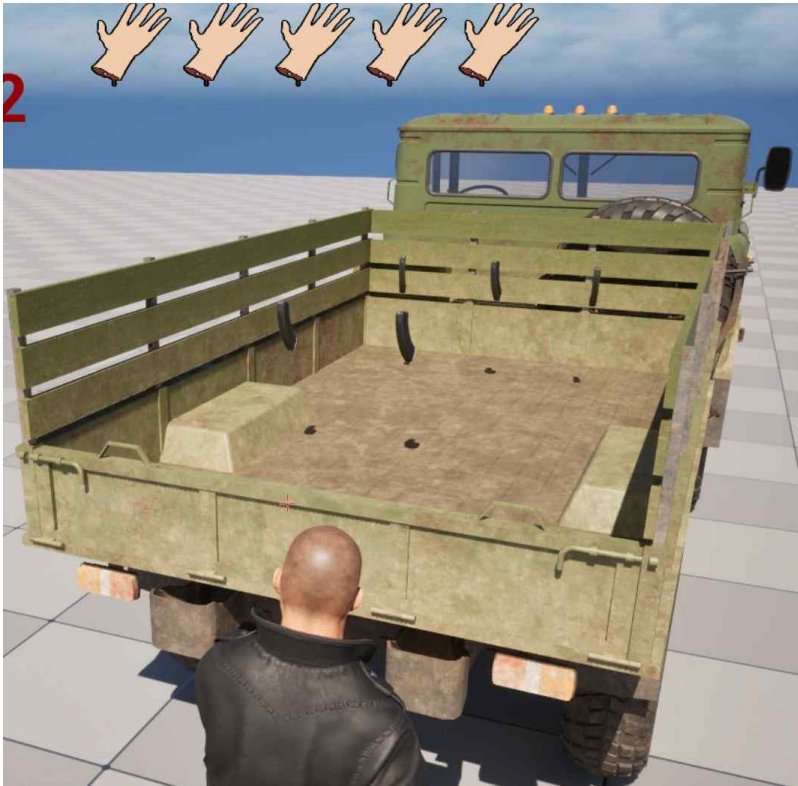


정상적으로 공격하는 모습을 확인할 수 있다.

이번 주 한 일

2.26 트럭 아이템 적재 모습 설정

기존에는 아래의 사진과 같이, 트럭에 아이템을 적재하면,
그 아이템의 메시가 일정거리 간격으로 동적으로 트럭위에 생성되도록 하였다.



하지만 동적으로 굳이 생성해야할 이유도 없고,
이쁘게 꾸미기도 힘들 것 같아서.
에디터에서 미리 이쁘게 메시를 쌓아두고 메시를 숨겨둔 뒤,
아이템이 적재되면 메시를 다시 보여주게 하는 방식으로 바꾸었다.

우선 종류별로 3개씩 컴포넌트로 만들어주었고.
에디터에서 위치와 매쉬를 설정할 수 있도록 해주었다

```
// AmmoSlots
for (int32 i = 0; i < 3; i++)
{
    FName SlotName = FName(*FString::Printf(TEXT("AmmoSlot_%d"), i));
    UStaticMeshComponent* NewSlot = CreateDefaultSubobject<UStaticMeshComponent>(SlotName);
    NewSlot->SetupAttachment(CargoOrigin);
    AmmoSlots.Add(NewSlot);
}

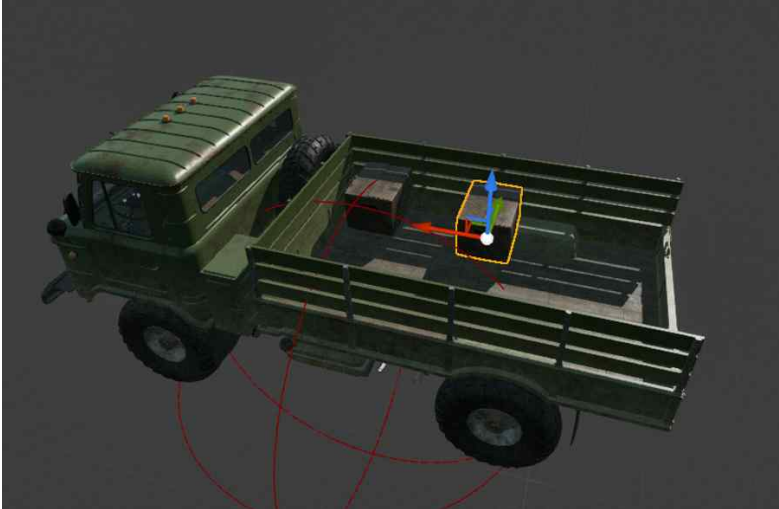
// FuelSlots
for (int32 i = 0; i < 3; i++)
{
    FName SlotName = FName(*FString::Printf(TEXT("FuelSlot_%d"), i));
    UStaticMeshComponent* NewSlot = CreateDefaultSubobject<UStaticMeshComponent>(SlotName);
    NewSlot->SetupAttachment(CargoOrigin);
    FuelSlots.Add(NewSlot);
}

// MedKitSlots
for (int32 i = 0; i < 3; i++)
{
    FName SlotName = FName(*FString::Printf(TEXT("MedKitSlot_%d"), i));
    UStaticMeshComponent* NewSlot = CreateDefaultSubobject<UStaticMeshComponent>(SlotName);
    NewSlot->SetupAttachment(CargoOrigin);
    MedKitSlots.Add(NewSlot);
}
```

Cargo Origin (CargoOrigin)		Edit in C++
	FuelSlot_1	Edit in C++
	FuelSlot_2	Edit in C++
	MedKitSlot_0	Edit in C++
	MedKitSlot_2	Edit in C++
	MedKitSlot_1	Edit in C++
	FuelSlot_0	Edit in C++
	AmmoSlot_2	Edit in C++
	AmmoSlot_1	Edit in C++
	AmmoSlot_0	Edit in C++

이번 주 한 일

아래와 같이 원하는 곳에 배치를 할 수 있고.



게임시작시엔 보이지않고, 아이템을 적재하면 모습이 보인다.

아이템 적재 전



적재 후



다음 주 할 일

- 캐릭터 애니메이션
- 아이템 관련 회의

특이사항

8, 9주 차 보고서

(2026년 2 월 15 일 ~ 2026년 2 월 28 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

```
// Overlapped 모델 (Completion Routine 콜백 기반)
// - 비동기 입출력 함수 완료되면, 쓰레드마다 있는 APC 큐에 일감이 쌓임
// - Alertable Wait 상태로 들어가서 APC 큐 비우기 (콜백 함수)
// 단점) APC 큐 쓰레드마다 있다! Alertable Wait 자체도 조금 부담!
// 단점) 이벤트 방식 소켓 : 이벤트 1:1 대응

// IOCP (Completion Port) 모델
// - APC -> Completion Port (쓰레드마다 있는 건 아니고 1개, 중앙에서 관리하는 APC 큐?)
// - Alertable Wait -> CP 결과 처리를 GetQueuedCompletionStatus
// 쓰레드랑 궁합이 굉장히 좋다!

// CreateIoCompletionPort
// GetQueuedCompletionStatus
```



```

enum IO_TYPE
{
    READ,
    WRITE,
    ACCPET,
    CONNECT,
};

struct OverlappedEx
{
    WSADOVERLAPPED overlapped = {};
    int32 type = 0; // read, write, accept, connect ...
};

void WorkerThreadMain(HANDLE iocpHandle)
{
    while (true)
    {
        DWORD bytesTransferred = 0;
        Session* session = nullptr;
        OverlappedEx* overlappedEx = nullptr;

        bool ret = ::GetQueuedCompletionStatus(iocpHandle, &bytesTransferred,
            (ULONG_PTR*)&session, (LPOVERLAPPED*)&overlappedEx, INFINITE);

        if (ret == FALSE || bytesTransferred == 0)
        {
            // TODO : 연결 끊김
            continue;
        }

        ASSERT_CRASH(overlappedEx->type == IO_TYPE::READ);

        cout << "Recv Data IOCP = " << bytesTransferred << endl;

        WSABUF wsBuf;
        wsBuf.buf = session->recvBuffer;
        wsBuf.len = BUFSIZE;

        DWORD recvLen = 0;
        DWORD flags = 0;
        ::WSARecv(session->socket, &wsBuf, 1, &recvLen, &flags, &overlappedEx->overlapped, NULL);
    }
}

```

```

vector<Session*> sessionManager;

// CP 생성
HANDLE iocpHandle = ::CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, 0, 0);

// WorkerThreads
for (int32 i = 0; i < 5; i++)
    GThreadManager->Launch([=]() { WorkerThreadMain(iocpHandle); });

// Main Thread = Accpet 담당
while (true)
{
    SOCKADDR_IN clientAddr;
    int32 addrLen = sizeof(clientAddr);

    SOCKET clientSocket = ::accept(listenSocket, (SOCKADDR*)&clientAddr, &addrLen);
    if (clientSocket == INVALID_SOCKET)
        return 0;

    Session* session = new Session();
    session->socket = clientSocket;
    sessionManager.push_back(session);

    cout << "Client Connected!" << endl;

    // 소켓을 CP에 등록
    ::CreateIoCompletionPort((HANDLE)clientSocket, iocpHandle, /*key*/(ULONG_PTR)session, 0);

    WSABUF wsaBuf;
    wsaBuf.buf = session->recvBuffer;
    wsaBuf.len = BUFSIZE;

    OverlappedEx* overlappedEx = new OverlappedEx();
    overlappedEx->type = IO_TYPE::READ;

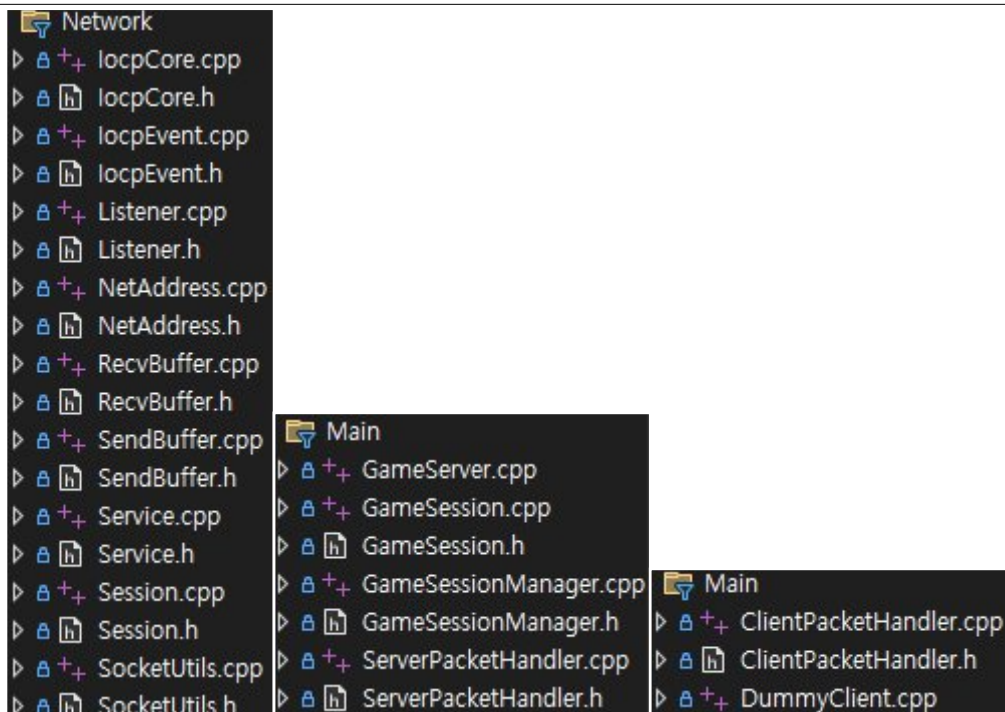
    DWORD recvLen = 0;
    DWORD flags = 0;
    ::WSARecv(clientSocket, &wsaBuf, 1, &recvLen, &flags, &overlappedEx->overlapped, NULL);
}

```

OverlappedEx 구조체는 WSAOVERLAPPED를 확장하기 위해 트릭으로 만든것인데 read인지 write인지 type 정보까지 꺼내 쓰기 위해 사용한다.

CreateIoCompletionPort를 호출해서 os에 I/O완료 통지를 받을 문을 만들고 WorkerThread를 실행하는 쓰레드를 미리 만들어 둔다. Accept로 클라이언트 소켓이 생성되면 이를 기존에 만든 iocpHandle에 등록한다. WSARecv를 호출해서 데이터가 들어오면 알려달라고 OS에 예약한다.

워커쓰레드는 GetQueueCompletionStatus함수에서 쓰레드가 대기 상태에 빠지고 I/O작업이 완료되면 OS가 이 쓰레드 중 하나를 깨운다. session에는 전에 등록한 CompletionKey가 돌아온다. 데이터를 다 처리했다면 다음 데이터를 받기 위해 다시 WSARecv를 호출한다. 이 과정이 반복되면서 끊임 없는 통신이 이루어지는 것이다.



이제 본격적으로 네트워크 라이브러리를 만들어 프레임워크 만들기를 시작했다. 어떤 이벤트가 발생했을 때, 그 일을 누구에게 시킬 것인가를 자동화해주는 식으로 만들었다.

locpcore, locpEvent, Listener, NetAddress, Session, SocketUtils 등의 파일을 만들었고, 각각 역할은 이렇다.

locpEvent

위에서 공부했던 OverlappedEx의 객체 지향 버전으로 OVERLAPPED 구조체를 상속받아서 OS가 읽을 수 있는 형태이면서 동시에 나만 아는 EventType정보를 담고 있다.

locpObject

이벤트가 완료되었을 때 실제 로직을 실행할 주체들의 부모이다. GetHandle()과 Dispatch()를 가진다. Listener는 새로운 손님이 오면 처리하고, Session은 손님이 보낸 데이터가 있으면 처리한다.

locpCore

가장 중요한 부분으로 Register는 소켓을 IOCP에 등록할 때 locpObject 자체를 Completion Key로 등록해버린다. 이렇게 하면 나중에 완료 통지가 올 때 이 소켓이 객체의 것이라는 걸 알 수 있다. Dispatch는 GetQueuedCompletionStatus를 계속 호출하다가 일이 완료되면, locpObject를 찾아 그 객체의 Dispatch()를 호출해준다.

Listener

기존의 accept()는 손님이 올때까지 스레드가 멈춰버리지만, 여기서는 AcceptEx라는 비동기 함수를 사용한다. RegisterAccept는 미리 알바생에서 손님이 오면 알려달라고 예약을 걸어두고, ProcessAccept는 손님이 오면 손님과 대화할 세션을 만들고, 중요한건 다시 RegisterAccept를 호출해서 다음 손님을 기다리게 한다.

Service :

서버 또는 클라이언트 전체를 관리하는 단위이다. ServerService는 Listener를 보유하며, 외부의 접속을 받아들이는 역할을 수행한다. ClientService는 서버에 접속을 시도하는 역할을 수행하며, 설정된 수만큼 세션을 생성해 연결을 시도한다. _sessions 셋을 통해 현재 연결된 모든 세션의 생명주기를 관리하며, SessionFactory를 통해 정의한 커스텀 세션 객체를 생성할 수 있게 한다.

RecvBuffer

수신된 데이터를 저장하는 순환 구조 형태의 버퍼이다. _readPos와 _writePos를 두어 데이터가 들어온 만큼 쓰고, 처리한 만큼 읽는다. 데이터가 버퍼 끝에 도달했을 때, 남은 데이터를 앞으로 복사하여 여유 공간을 확보한다. OVERLAPPED 구조체를 상속받아 비동기 작업 완료 시 어떤 이벤트인지 구분하고 관련 데이터를 보관하는 역할을 한다.

SendBuffer

메모리 할당 효율을 높이고 파편화를 방지하게 위해 설계한 청크 기반 메모리 관리 시스템이다.

SendBufferChunk를 크게 하나 할당해서 필요한 만큼 잘라쓰고, 다 쓰면 다시 재활용하는 방식이다. SendBuffer는 실제로 데이터를 담는 인터페이스이다. SendBufferChunk의 특정 영역을 가리키는 포인터를 가지고 있다. 사용이 끝나면 Close()를 호출하여 사용한 실제 크기를 기록한다.

SendBufferChunk는 엄청 큰 크기의 실제 메모리 배열을 들고 있다. 하나의 청크 안에서 여러 개의 SendBuffer가 순차적으로 할당된다. _usedSize변수를 통해 현재 어디까지 사용했는지 관리하며, 다쓰면 Reset()을 통해 재사용된다.

SendBufferManager는 SendBufferChunk들을 리스트로 관리한다. 쓰레드 세이프하게 설계하여 여러 쓰레드가 동시에 메모리를 요청해도 안정하게 청크를 빌려주고 회수한다.

기존의 Session은 단순히 바이트 덩어리를 주고 받았는데 PacketSession을 추가하여 그 데이터가 의미 있는 단위인 패킷으로 조립하는 역할을 하게 했다. 패킷의 전체 크기와 패킷 종류를 패킷 헤더 구조체 안에 넣었다. 그리고 OnRecv를 재정의하여 최소한 헤더만큼은 데이터가 왔는지 확인하고, 헤더를 읽어서 패킷의 전체 크기를 파악하고, 전체 패킷 크기만큼 데이터가 다 왔는지 확인 후 패킷이 완성되면 OnRecvPacket을 호출한다. PacketSession은 OnRecv를 sealed로 막아버리고 대신 OnRecvPacket이라는 가상 함수를 사용한다.

BufferReader와 BufferWriter를 추가하여 메모리 버퍼에 데이터를 직렬화 하거나 역직렬화 할 때 사용하는 유틸리티 도구로 만들었다. 네트워크 패킷을 만들거나 파일을 저장할 때, 여러 데이터를 하나의 바이트 덩어리로 묶고 푸는 역할을 편리하게 도와주는 역할을 한다.

BufferWriter는 특정 메모리 공간에 순차적으로 데이터를 기록하는 역할을 한다. operator<<는 정수나 구조체 같은 데이터를 writer << value; 형태로 쉽게 버퍼에 넣을 수 있다. Reserve<T>()는 데이터를 바로 쓰지 않고, 나중에 값을 채우기 위해 특정 크기만큼의 공간을 미리 확보하고 그 주소를 반환한다. 데이터를 쓰기 전에 FreeSize()를 체크해서 버퍼 오버플로우를 방지한다.

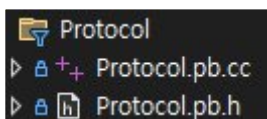
BufferReader는 버퍼에 담긴 바이트 데이터를 다시 의미 있는 데이터 타입으로 읽어오는 역할을 한다. operator>>는 reader >> value; 형태로 버퍼의 데이터를 순서대로 꺼내온다. Peek()는 현재 읽는 위치를 옮기지 않고 데이터만 살짝 확인한다. Read()는 데이터를 읽어오면서 동시에 읽는 위치를 뒤로 이동시킨다.

서버와 클라이언트에 PacketHandler를 만들어주었다.

ServerPacketHandler는 서버에서 클라이언트로 데이터를 보낼 패킷을 조립한다. Make_S_TEST함수는 GSendBufferManager를 통해 빈 버퍼를 할당받는다. BufferWriter를 이용해 ID->HP->공격력 순서로 데이터를 밀어 넣는다. 버퍼 리스트의 경우 buffs.size()를 먼저 기록하여 클라이언트가 몇 개를 읽어야 할지 알려준 뒤, 루프를 돌며 세부 정보를 기록한다. 최종적으로 패킷 전체 크기를 헤더에 기록하고 SendBuffer를 반환한다.

ClientPacketHandler는 서버로부터 받은 바이너리 덩어리를 해석한다. HandlePacket은 패킷 헤더를 먼저 읽어 ID를 확인하고, 해당 ID에 맞는 처리함수로 분기한다. Handle_S_TEST는 BufferReader를 사용하여 서버가 넣은 순서 그대로 데이터를 뽑아낸다. buffCount를 읽어온 뒤, vector::resize()를 통해 공간을 확보하고 루프를 돌며 버퍼 정보를 채운다. 그리고 cout해서 수신된 데이터를 화면에 출력하여 확인한다.

현재는 패킷의 종류가 하나뿐이라 switch문이 단순하지만 패킷 종류가 여러 개가 되면 이 방식은 관리가 힘들어진 다. 후에 Google Protocol Buffer를 사용할 예정이라서 일단은 문제가 있는 방식이라도 이렇게 연습해보았다.



Google Protocol Buffers를 프로젝트에 추가해보았다. 구글에서 만든 데이터를 효율적으로 주고받기 위한 규격이다. 보통 JSON이나 XML을 사용하지만 용량이 크고 파싱하는데 에너지가 많이 든다. Protobuf는 이를 해결하기 위해 데이터를 이진 형태로 압축한다.

다음 주 할 일

패킷 자동화와 JobQueue에 대해서 공부를 할 예정이다. 데이터베이스를 연동하는 것도 연습해볼것이다.

특이사항

9주 차 보고서

(2026년 2 월 15 일 ~ 2026년 2 월 28 일)

작성자

안윤진

이번 주 한 일

- PCG로 만들어둔 집 액터들을 랜덤 세팅하려함. 그러나 static mesh spawner로 기존에 해왔던 방식과 다르게 블루프린트로 만든 액터는 spawn actor로 세팅이 되지 않는 문제 발생.

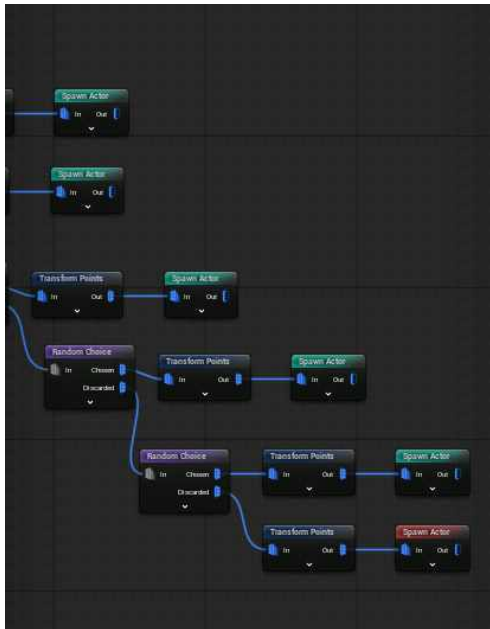
왜인지 모르겠으나 다른 날 시도하니 잘 됨. spawn actor에서 옵션은 노 머지로 선택 -> 각각 개별 블루프린트로 구성되어있고 후에 각자 상호작용도 필요해서 합치지 않음,

attach 옵션 - 생성된 액터를 pcg volume이나 컴포넌트에 붙일지 월드에 독립적으로 돌지 선택하는 요소.

not - 볼륨 이동 시에도 위치가 크게 변하지 않는 이상 따라가 변하지 않음

attach - 볼륨 이동에 따라 다시 랜덤 배치됨. 이때 볼륨 스케일 조절을 했을 때 집 액터들의 스케일은 다시 원상복구되어 위치 변화말고 고려할 점은 없다 판단. 현재 attach로 설정.

+



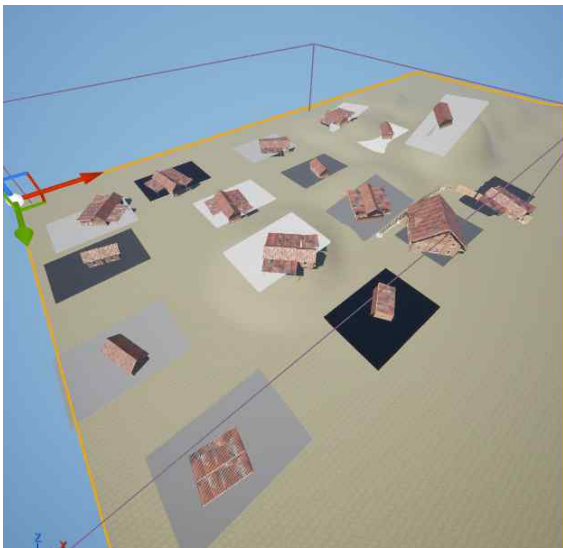
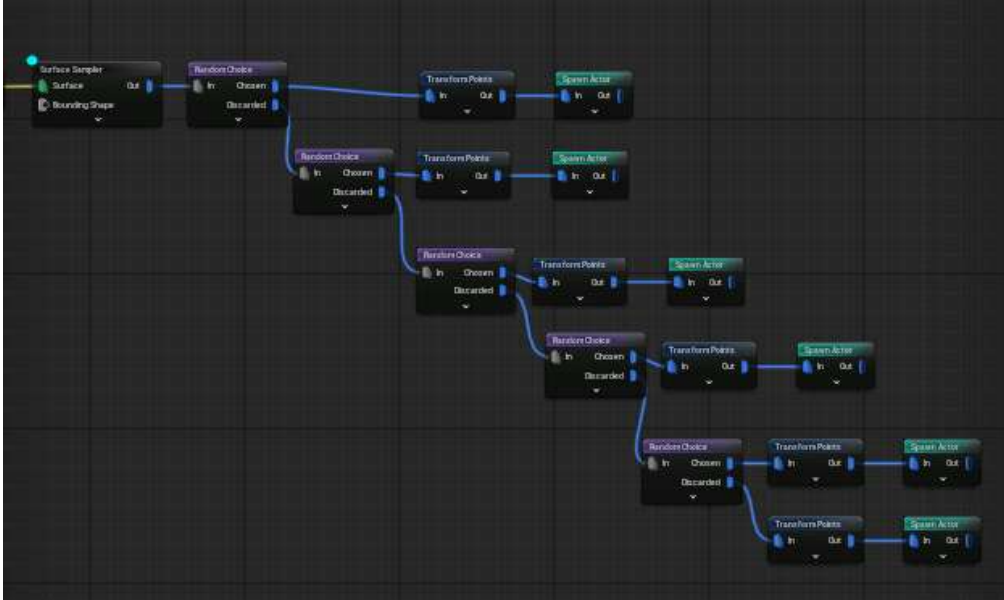
그려지지 않는 액터가 존재하는 걸 찾음. 노드의 색이 다르게 표시되었는데, 빨간 노드인 경우 BP_HOUSE1으로 혼자 전에 작성했던 다른 class 기반 블루프린트였다. 실행은 잘 되어도 PCG가 "스폰 실패"로 판단하면 빨간색으로 표시된다. 그래서 코드를 다 비교해보았지만 문제를 못 찾던 도중 비슷하게 설정해 놓은 다른 액터 클래스 문제란걸 깨달았다. 이름 세팅과 필요없어진 경우 헛갈리지 않게 지우던가 이름 수정하는.. 정리의 중요성을 느꼈다.

- 위에 문제에서 더 파고들어, 배치 성공했더니 이젠 종류 랜덤 세팅에서 막힘. static mesh 같은 경우 기본 세팅에 여러 메쉬들을 추가하고 가중치를 통해 랜덤값을 조절하는 기능이 있었으나, spawn actor의 경우 하나의 클래스만 지정 가능.

attribute를 통해 랜덤세팅을 한다는 글은 몇 개 보았지만, 이해할 수 있는 자료를 찾기엔 어려웠음.

이번 주 한 일

머릿속 이론으로 생각한 바가 있으나 언리얼에 있는 노드를 찾아서 하려니 제대로 알지 못해 시간이 좀 걸렸다. 애트리뷰트를 사용하는 방법을 익히지 못해 랜덤 값으로 퍼센티지를 정한 후 이걸 반복해서 적용하였다. n%에 해당하면 a 스폰, 해당x -> n%에 해당 -> b스폰, 해당x-> n%..이런식으로 나머지는 다른 액터 생성을 통해 정해진 범위안에 집들을 랜덤배치 하였다. 이때 이 집들은 랜덤 스케이프 데이터를 가져와 제작하였기에 바닥의 굴곡에도 나름 맞춰 생성된다.



원래 구는
playercontroller에 넣는게 맞는데.
UE5부터 pawn에 움직이는 행동은 넣는게 규행이라는 듯함.
pawn이 움직이는 거니까. 신형도 이 자료들보고 한 듯하다.
내가 컨트롤러 따로 만들면서 실행하려 했으나, 다른 건 다 움직이지만
마우스 움직임에 따른 시야 회전을 찾지 못했다. 요즘 다 이런 식인
자료 밖에 없었다. 그래서 원상복구 해두고 그냥 HUD 연결에만
작기로 변경한 역할을 축소했다.

- UI

플레이어 컨트롤러로 역할을 전부 옮기려다 위의 이유들로 그냥 다시 원상복구하고, 위젯들과 새로 추가할 기능(위젯과 연결된 기능)만 플레이어 컨트롤러에 넣기로 했다.

이번 주 한 일

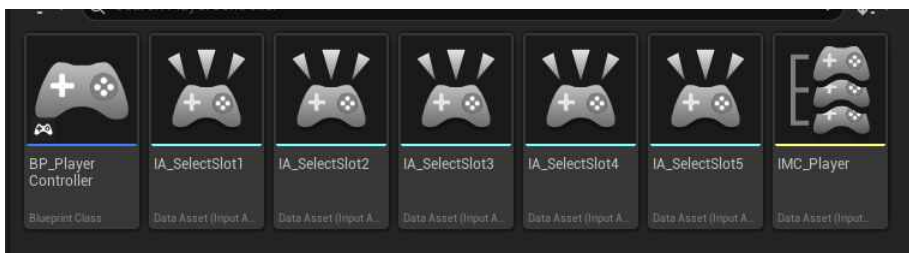
```
void AFPSPlayerController::BeginPlay()
{
    Super::BeginPlay();
    if (!InvenWidgetClass || !TimerWidgetClass) return;

    InventoryW = CreateWidget<UInventoryWidget>(this, InvenWidgetClass);
    TimerW = CreateWidget<UBaseUI>(this, TimerWidgetClass);
    //여기서 위젯 생성 후 플레이어에서 뷰포트에 추가함. 여기서 뷰포트 추가하면
    //순서때문에 화면에 안그려짐.

    //입력 시스템 등록
    UEnhancedInputLocalPlayerSubsystem* Subsystem =
        ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(GetLocalPlayer());

    if (Subsystem) {
        Subsystem->AddMappingContext(PlayerMappingContext, 0);
    }
}
```

컨트롤러에서 바로 그리려했으나 화면에 그려지지 않기에 뷰포트에 위젯을 추가하는 것은 pawn에서 위젯이 다 생성된 후에 그리기로 결정.



그 외 슬롯을 선택할 수 있는 숫자키들을 추가. 키 누를 시 깜빡이는 애니메이션 진행으로 현재 어느 키가 눌러있는지 확인 가능.



제작해둔 타이머와 슬롯(인벤토리) 위젯을 합쳤다.

- 합치는 과정에서 기존 다른 파일에 만들어둔 class와 위젯블루프린트를 가져왔는데, 이때 경로 문제가 생겼다. class는 열려도 그 기반으로 만든 블루프린트는 열리지 않는 문제가 발생.

```
FPSPProject\Config\DefaultEngine.ini

@@ -152,3 +152,5 @@ ManualIPAddress=
152 152 +CollisionChannelRedirects=(OldName="VehicleMovement",NewName="Vehicle")
153 153 +CollisionChannelRedirects=(OldName="PawnMovement",NewName="Pawn")
154 154
155 + [/Script/Engine.Engine]
156 + +ActiveClassRedirects=(OldClassName="/Script/Niagara_Test.BaseUI",NewClassName="/Script/FPSPProject.BaseUI")
```

그래서 ini 파일에 저렇게 변경된 경로를 추가해주었다. 기존 경로를 적어주고 변경된 경로를 적어주니 해결되었다. 이부분은 잘 몰라서 지파티와 함께했다.

그 후 머지했다.

다음 주 할 일
집마다 상호작용 가능한 문 달고, 이제 정말 맵 끝내기..

특이사항